GRUPO 5 · REQUERIMIENTOS FUNCIONALES

Tarjetas de Crédito y Débito

Módulo: Gestión de tarjetas

Curso de Automatización de Pruebas de Software

Universidad Nacional de Asunción — CIT

Profesor: Steven Gracia

Campo	Valor
Documento	Requerimientos funcionales — Grupo 5
Versión	1.0
Fecha	22 de agosto de 2026
Sistemas alcanzados	aiquaa-sandbox-api (REST) y aiquaa-sandbox-web (front)
Fuente de la lógica	Código fuente de ambos repositorios y scripts SQL del schema qa_training
Uso previsto	Base para escribir escenarios BDD/Gherkin y casos de prueba automatizados

> Documento derivado del código fuente de aiquaa-sandbox-api y aiquaa-sandbox-web. Describe el comportamiento realmente implementado en el sandbox de práctica, no un sistema bancario productivo. El anexo normativo es material de referencia orientativo.

Contenido

1. Alcance y actores
2. Precondiciones y acceso
3. Modelo de datos del módulo
4. Requerimientos funcionales
5. Reglas transversales de la API
6. Flujo en la aplicación web
7. Criterios de aceptación
8. Anexo normativo (BCP / SEDECO) y brechas

1. Alcance y actores

Cubre el CRUD de tarjetas emitidas a un titular y el ciclo de bloqueo y desbloqueo. El número de tarjeta que devuelve el sistema es siempre un número enmascarado: el sandbox no guarda datos reales de tarjeta.

Fuera de alcance: Consumos y autorizaciones de compra, cálculo de límites disponibles, fecha de expiración, renovación, reposición por robo o extravío y estado de cuenta: ninguna de estas operaciones existe en el sandbox.

Actores

Actor	Descripción
Alumno / QA automatizador	Consume la API con su propia API key y automatiza escenarios BDD sobre el módulo de su grupo.
Usuario de negocio (fila de `usuarios`)	Identidad de dominio sobre la que operan los endpoints: es dueño de cuentas, facturas, tarjetas, órdenes, reservas, notificaciones y roles.
API REST (aiquaa-sandbox-api)	Ejecuta la lógica de negocio con SQL fijo bajo el rol de base de datos `qa_api` (SELECT + INSERT + UPDATE, sin DELETE).
Aplicación web (aiquaa-sandbox-web)	Interfaz que consume la API a través del proxy `/api/proxy/{path}`; nunca llama al backend directamente desde el navegador.

Endpoints del módulo

Método y ruta	Requerimiento	Descripción
GET /api/v1/tarjetas	RF-G5-01	Lista tarjetas activas, opcionalmente filtradas por titular.
POST /api/v1/tarjetas	RF-G5-02	Emite una tarjeta nueva, activa desde el momento de la emisión.
GET /api/v1/tarjetas/{id}	RF-G5-05	Devuelve el detalle de una tarjeta.
PUT /api/v1/tarjetas/{id}	RF-G5-06	Reemplaza tipo, marca y límite de crédito de una tarjeta.
DELETE /api/v1/tarjetas/{id}	RF-G5-07	Da de baja (soft-delete) una tarjeta.
PATCH /api/v1/tarjetas/{id}/bloquear	RF-G5-03	Bloquea una tarjeta.
PATCH /api/v1/tarjetas/{id}/activar	RF-G5-04	Activa (o reactiva) una tarjeta.

2. Precondiciones y acceso

- Toda request a la API debe enviar el header `x-api-key` con una clave existente y con `active = true` en la tabla `public.api_keys`; sin header o con clave inválida/inactiva la respuesta es `401 UNAUTHORIZED` con el mensaje genérico "Invalid or inactive API key.".
- Cada API key admite un máximo de **200 requests por minuto** (ventana deslizante). Superado el límite la API responde `429 RATE_LIMITED` con los headers `Retry-After`, `X-RateLimit-Limit`, `X-RateLimit-Remaining` y `X-RateLimit-Reset`.

- Los datos de práctica viven en el schema aislado `qa_training` y se cargan con `scripts/seed-data.sql`; la data es determinística, por lo que un escenario puede apoyarse en los registros sembrados.
- La API no expone borrado físico: el rol `qa_api` no tiene permiso `DELETE` en Postgres, y todo endpoint `DELETE` de la API hace baja lógica (`activo = false`), devolviendo `204 No Content` sin cuerpo.
- Casi todas las tablas de negocio (`sesiones`, `transferencias`, `facturas`, `tarjetas`, `notificaciones`, `ordenes`, `reservas`, `movimientos`, `roles`, además de `usuarios` y `cuentas` que ya la tenían) tienen columna `activo` / `activa`; todo `GET`, `PUT` y `DELETE` de esa tabla filtra u opera solo sobre filas con `activo = true` — una fila dada de baja deja de ser visible y de poder reemplazarse, pero no se borra.
- En la aplicación web el acceso tiene dos capas: capa 1 la API key (pantalla `/login`, se valida contra `GET /api/v1/roles` antes de guardarse) y capa 2 el usuario de negocio (pantalla `/auth/login`). Con `NEXT_PUBLIC_DEMO_MODE=true` la capa 1 se omite y el proxy inyecta una key demo del servidor.
- Las rutas `/usuarios/*` de la web son la excepción al guard de capa 2: se pueden usar solo con la API key, porque son el punto de partida para obtener un `usuarioId`.
- La emisión requiere un titular existente; el `usuarioId` se obtiene del alta de cliente (RF-G4-01) o de los datos sembrados.
- Los datos sembrados incluyen tarjetas en los tres estados posibles (`activa`, `bloqueada`, `vencida`), útiles para los escenarios de transición.

3. Modelo de datos del módulo

tarjetas

Plásticos emitidos a los titulares. Es la única tabla que toca el módulo.

Columna	Tipo / restricción
`id`	`bigserial`, clave primaria
`usuario_id`	`bigint`, obligatorio, referencia a `usuarios(id)`
`tipo`	`text`, `credito` o `debito`; por defecto `debito`
`marca`	`text`, `visa` o `mastercard`; por defecto `visa`
`numero_enmascarado`	`text`, obligatorio; formato `** ** NNNN`
`limite_credito`	`numeric(12,2)`, opcional; la emisión por API lo deja vacío
`saldo_actual`	`numeric(12,2)`, obligatorio, por defecto 0
`estado`	`text`, uno de `activa` / `bloqueada` / `vencida`; por defecto `activa`
`created_at`	`timestamptz`, por defecto `now()`
`activo`	`boolean`, obligatorio, por defecto `true`; lo usan `GET/PUT/DELETE` para filtrar/operar

> Los cuatro últimos dígitos del número enmascarado se generan al azar en cada emisión: no son estables entre ejecuciones y ningún escenario debe esperar un valor concreto. Sí es estable el formato `** **` seguido de cuatro dígitos.

4. Requerimientos funcionales

RF-G5-01 — Listar tarjetas

`GET /api/v1/tarjetas`

Devuelve las tarjetas del sandbox, con la posibilidad de acotar el resultado a un titular determinado.

Entradas y validaciones

- `usuarioId` (opcional, por query): número entero positivo.

Reglas de negocio

- Devuelve únicamente tarjetas con `activo = true`; una dada de baja con RF-G5-07 deja de aparecer.
- Con `usuarioId`, devuelve todas las tarjetas activas de ese titular ordenadas por identificador ascendente.
- Sin `usuarioId`, devuelve las primeras 100 tarjetas activas del sandbox.
- El listado incluye tarjetas en cualquier `estado` de negocio (activa/bloqueada/vencida): no filtra por eso, solo por `activo`.
- Un titular sin tarjetas devuelve una lista vacía.

Respuesta esperada

- `200 OK` con `data` como arreglo de tarjetas, cada una con `id`, `usuario_id`, `tipo`, `marca`, `numero_enmascarado`, `limite_credito`, `saldo_actual`, `estado` y `created_at`.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	`usuarioId` presente pero no numérico, cero o negativo.
`UNAUTHORIZED`	401	Falta la API key o es inválida.

Fuente: `app/api/v1/tarjetas/route.ts`

RF-G5-02 — Emitir una tarjeta

`POST /api/v1/tarjetas`

Emite una tarjeta nueva para un titular, del tipo y la marca indicados. La tarjeta queda activa y operativa desde su emisión.

Entradas y validaciones

- `usuarioId` (obligatorio): número entero positivo del titular.
- `tipo` (obligatorio): `credito` o `debito`.
- `marca` (obligatorio): `visa` o `mastercard`.

Reglas de negocio

- La tarjeta se emite siempre con `estado = 'activa'`; el cliente no puede elegir el estado inicial.
- El número enmascarado lo genera el sistema con el formato `** ** NNNN`, donde los cuatro dígitos finales son aleatorios.
- El saldo inicial es cero y el límite de crédito queda vacío, incluso para tarjetas de tipo `credito`.
- El titular debe existir: un `usuarioId` inexistente falla como error de ejecución por violación de clave foránea.
- **No hay tope de tarjetas por titular ni control de duplicados:** el mismo titular puede emitir cuantas tarjetas quiera, del mismo tipo y marca.
- No se verifica el estado de KYC del titular antes de emitir.

Respuesta esperada

- `201 Created` con `data` conteniendo la tarjeta emitida.

Errores

Código	HTTP	Cuándo ocurre
`VALIDATION_ERROR`	400	Falta un campo obligatorio, `tipo`/`marca` tienen un valor fuera de la lista permitida, o el titular indicado no existe.

Fuente: `app/api/v1/tarjetas/route.ts`

RF-G5-05 — Consultar una tarjeta

`GET /api/v1/tarjetas/{id}`

Devuelve el detalle de una tarjeta puntual.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Devuelve la tarjeta cuyo identificador coincide exactamente, siempre que siga `activo = true`.
- Si no existe o ya está dada de baja, responde 404 con el mensaje "Tarjeta no encontrada.".

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta completa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/tarjetas/[id]/route.ts`

RF-G5-06 — Reemplazar una tarjeta

`PUT /api/v1/tarjetas/{id}`

Reemplaza el tipo, la marca y, opcionalmente, el límite de crédito de una tarjeta existente. Es la única forma de asignarle un límite de crédito, algo que la emisión (RF-G5-02) no permite.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.
- `tipo` (obligatorio): `credito` o `debito`.
- `marca` (obligatorio): `visa` o `mastercard`.
- `limiteCredito` (opcional): número mayor que cero.

Reglas de negocio

- Solo puede reemplazar una tarjeta con `activo = true`; una dada de baja responde 404.
- `numeroEnmascarado` y `saldoActual` no forman parte del cuerpo: quedan fuera del control del cliente.
- `estado` tampoco forma parte del cuerpo: bloquear/activar sigue siendo exclusivo de RF-G5-03/04.
- Omitir `limiteCredito` lo deja vacío (`null`), aunque la tarjeta ya tuviera uno asignado: es un reemplazo completo, no un parche.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta ya actualizada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`VALIDATION_ERROR`	400	Falta un campo obligatorio, o tiene un valor fuera de la lista permitida.

Fuente: `app/api/v1/tarjetas/[id]/route.ts`

RF-G5-07 — Dar de baja una tarjeta

`DELETE /api/v1/tarjetas/{id}`

Da de baja lógica una tarjeta, quitándola de los listados y consultas por id.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo.

Reglas de negocio

- Marca `activo = false`; la fila permanece en la base, nunca se borra físicamente.
- Es distinta de bloquear (RF-G5-03): bloquear cambia el `estado` de negocio mientras la tarjeta sigue existiendo para el CRUD; dar de baja la saca del CRUD sin importar su `estado` de negocio.
- Una tarjeta ya bloqueada o vencida también puede darse de baja.

Respuesta esperada

- `204 No Content`, sin cuerpo.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta vigente con ese identificador.
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/tarjetas/[id]/route.ts`

RF-G5-03 — Bloquear una tarjeta

`PATCH /api/v1/tarjetas/{id}/bloquear`

Deja una tarjeta fuera de servicio, por ejemplo ante un extravío. El bloqueo es inmediato y no requiere motivo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la tarjeta.

Reglas de negocio

- El estado de la tarjeta pasa a `bloqueada` sin verificar el estado anterior.
- La operación es idempotente: bloquear una tarjeta ya bloqueada devuelve el mismo resultado, sin error.
- También puede bloquearse una tarjeta en estado `vencida`.
- No se registra motivo, ni fecha de bloqueo distinta de la de la bitácora de requests, ni quién lo solicitó.
- Si la tarjeta no existe, responde 404 y no modifica nada.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta ya bloqueada.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta con ese identificador (mensaje: "Tarjeta no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/tarjetas/[id]/bloquear/route.ts`

RF-G5-04 — Activar una tarjeta

`PATCH /api/v1/tarjetas/{id}/activar`

Vuelve a poner en servicio una tarjeta, revirtiendo un bloqueo previo.

Entradas y validaciones

- `id` (obligatorio, en la ruta): número entero positivo de la tarjeta.

Reglas de negocio

- El estado de la tarjeta pasa a `activa` sin verificar el estado anterior.
- La operación es idempotente: activar una tarjeta ya activa devuelve el mismo resultado.
- **Una tarjeta en estado `vencida` también puede activarse**, lo que en un sistema real requeriría reemisión del plástico.
- El bloqueo y la activación pueden alternarse indefinidamente sin límite de intentos.
- Si la tarjeta no existe, responde 404 y no modifica nada.

Respuesta esperada

- `200 OK` con `data` conteniendo la tarjeta ya activa.

Errores

Código	HTTP	Cuándo ocurre
`NOT_FOUND`	404	No existe una tarjeta con ese identificador (mensaje: "Tarjeta no encontrada.").
`VALIDATION_ERROR`	400	El identificador no es un entero positivo.

Fuente: `app/api/v1/tarjetas/[id]/activar/route.ts`

5. Reglas transversales de la API

Todas las rutas REST comparten el mismo pipeline: autenticación → control de caudal → validación de entrada → ejecución del SQL fijo → auditoría. Estas reglas aplican a cada requerimiento del documento y no se repiten en cada uno.

- **Envoltura de respuesta:** las respuestas exitosas devuelven `{ "data": ... }` y las fallidas `{ "error": { "code", "message", "details?" } }`.
- **Validación de entrada:** cada ruta define un esquema; los parámetros de query, el cuerpo JSON y los parámetros de ruta se combinan en un único objeto con precedencia query < body < path (un `{id}` de la URL siempre gana sobre un `id` del cuerpo).
- **Coerción de tipos:** los identificadores y montos que llegan por query o por path se convierten de texto a número automáticamente; los campos numéricos de un cuerpo JSON deben enviarse como número, no como texto, salvo que el requerimiento indique lo contrario.
- **Cuerpo JSON malformado:** un cuerpo que no parsea como JSON devuelve `400 VALIDATION_ERROR` con el mensaje "Invalid JSON body.".
- **Errores de base de datos, mapeados por código SQLSTATE:** una violación de unicidad (`23505`, ej. un email o número de factura repetido) se traduce a `409 CONFLICT`; una violación de clave foránea o de restricción `CHECK` (`23503` / `23514`) se traduce a `400 VALIDATION_ERROR`; cualquier otro error de base de datos cae en `400 EXECUTION_ERROR`.

- `DELETE` **es siempre baja lógica:** actualiza `activo = false` (o `activa` en `cuentas`) y responde `204 No Content` **sin cuerpo** — no `200` con la fila afectada.
- **Verbo no soportado:** una ruta que no define el método HTTP usado responde `405 Method Not Allowed`; lo genera Next.js automáticamente, ninguna ruta lo arma a mano, y su cuerpo no sigue la envoltura `{ error }` del resto de la API.
- **Auditoría:** toda request, exitosa o fallida, deja un registro en `public.sql_audit_log` con la clave usada, el método y la ruta, la entrada validada, el resultado y la IP de origen. Un fallo de auditoría nunca convierte una respuesta exitosa en error.
- **Idempotencia de las transiciones de estado:** los endpoints que fijan un estado (`bloquear`, `activar`, `confirmar`, `cancelar`, `leer`) escriben el valor destino sin verificar el estado previo, por lo que repetir la llamada devuelve el mismo resultado.
- **Las transiciones de estado anteriores al CRUD no miran `activo`:** `PATCH .../bloquear`, `.../activar`, `.../leer`, `.../confirmar`, `.../cancelar`, `.../kyc` y `POST .../pagar` no filtran por `activo = true` (ninguno de esos archivos se tocó al agregar el CRUD). Un recurso dado de baja con el nuevo `DELETE` deja de aparecer en `GET`, pero **sigue pudiendo bloquearse, confirmarse, leerse o pagarse** a través de su acción dedicada — una inconsistencia real del sandbox, no un comportamiento a asumir como intencional.

Códigos de éxito

Status	Cuándo ocurre
`200 OK`	GET/PUT/PATCH exitoso; también un POST de upsert que reactivó una fila existente en vez de crearla.
`201 Created`	POST que crea una fila nueva.
`204 No Content`	DELETE exitoso (baja lógica) — la respuesta no lleva cuerpo.

Códigos de error

Código	HTTP	Significado
`UNAUTHORIZED`	401	Falta el header `x-api-key`, o la clave no existe o está inactiva.
`RATE_LIMITED`	429	Se superaron las 200 requests por minuto de esa API key.
`VALIDATION_ERROR`	400	La entrada no cumple el esquema de la ruta, el JSON es inválido, o Postgres rechazó una clave foránea/CHECK.
`EXECUTION_ERROR`	400	La consulta falló en la base de datos por un motivo no cubierto por los códigos anteriores.
`NOT_FOUND`	404	El recurso indicado no existe, ya está dado de baja (`activo=false`), o no está en un estado que admita la operación.
`CONFLICT`	409	Violación de restricción `UNIQUE` en Postgres (ej. email, número de factura o nombre de rol duplicado).
`INTERNAL_ERROR`	500	Fallo inesperado del servidor (por ejemplo, no se pudo verificar la API key).

6. Flujo en la aplicación web

La aplicación web consume exactamente los mismos endpoints a través del proxy `/api/proxy/{path}`, que agrega la API key del lado del servidor y reenvía los headers de límite de caudal. El menú de navegación muestra únicamente los módulos del grupo del alumno cuando su email figura en el roster del curso; si no figura, muestra los diez módulos. Los errores de la API se muestran en pantalla con el mensaje que devolvió el backend, y un `401` cierra la sesión local. **La interfaz web todavía no tiene pantallas para `PUT` / `DELETE` ni para los recursos nuevos (`sesiones`, `movimientos`)**: solo cubre los flujos de creación y consulta que ya existían; esos requerimientos se prueban llamando la API directamente (Postman, `curl`, o el cliente HTTP del framework de automatización), no navegando la web.

Pantalla	Qué permite hacer
`/tarjetas` — Listado de tarjetas	Tabla con tipo, marca, número enmascarado, saldo y estado, con filtro por titular y botones de bloqueo y activación por fila. Ejecuta RF-G5-01, RF-G5-03 y RF-G5-04.
`/tarjetas/new` — Emisión de tarjeta	Formulario con titular, tipo y marca. Ejecuta RF-G5-02 y, al emitirse, vuelve al listado.

- Los botones de acción de cada fila refrescan el listado tras la operación, de modo que el nuevo estado se ve sin recargar la página.
- El campo de titular del formulario se completa por defecto con el usuario en sesión, pero puede cambiarse a mano.
- No hay pantalla de detalle, edición ni baja individual: `GET`, `PUT` y `DELETE /tarjetas/{id}` (RF-G5-05/06/07) solo se pueden ejercitar llamando la API directamente.

7. Criterios de aceptación

Criterios de aceptación redactados en prosa, uno o más por requerimiento, cubriendo el camino feliz y los caminos negativos. Son la base para derivar escenarios BDD.

RF-G5-01 — Listar tarjetas

- Al listar tarjetas filtrando por un titular, todos los elementos devueltos tienen ese `usuario_id`.
- El listado incluye tarjetas bloqueadas y vencidas junto a las activas.
- Al filtrar por un titular sin tarjetas, la respuesta es 200 con `data` vacío.
- Ninguna tarjeta expone un número completo: todas devuelven el formato enmascarado.

RF-G5-02 — Emitir una tarjeta

- Al emitir una tarjeta para un titular existente, la respuesta es 201, `data.estado` es `activa` y `data.saldo_actual` es cero.
- El `numero_enmascarado` devuelto empieza con `** **` y termina en cuatro dígitos.
- Al emitir una tarjeta de crédito, `limite_credito` queda vacío: la emisión no asigna límite.
- Al emitir dos tarjetas seguidas para el mismo titular con el mismo tipo y marca, ambas respuestas son 201 y se crean dos tarjetas distintas.
- Al emitir con `marca=amex`, la respuesta es 400 `VALIDATION_ERROR`.

- Al emitir para un `usuarioId` inexistente, la respuesta es 400 `VALIDATION_ERROR` y no se crea ninguna tarjeta.
- Al emitir para un titular con KYC pendiente, la operación es aceptada: la emisión no consulta el estado de verificación.

RF-G5-05 — Consultar una tarjeta

- Dada una tarjeta existente, al consultarla por su `id` la respuesta es 200 y los datos coinciden con los de la emisión.
- Al consultar una tarjeta inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G5-06 — Reemplazar una tarjeta

- Dada una tarjeta de crédito sin límite, al reemplazarla enviando `limiteCredito`, la respuesta es 200 y `data.limite_credito` refleja el nuevo valor.
- El `estado` de la tarjeta no cambia por este reemplazo.
- Al reemplazar sin enviar `limiteCredito` una tarjeta que ya tenía uno asignado, el límite queda vacío.
- Al reemplazar una tarjeta inexistente o dada de baja, la respuesta es 404 `NOT_FOUND`.

RF-G5-07 — Dar de baja una tarjeta

- Dada una tarjeta vigente, al darla de baja la respuesta es 204 sin cuerpo.
- Tras la baja, `GET /tarjetas/{id}` sobre esa tarjeta responde 404, y deja de aparecer en RF-G5-01.
- Dar de baja una tarjeta bloqueada es aceptado igual (204).

RF-G5-03 — Bloquear una tarjeta

- Dada una tarjeta activa, al bloquearla la respuesta es 200 y `data.estado` es `bloqueada`.
- Al bloquear dos veces la misma tarjeta, ambas respuestas son 200 y el estado final sigue siendo `bloqueada`.
- Al bloquear una tarjeta inexistente, la respuesta es 404 `NOT_FOUND` con el mensaje "Tarjeta no encontrada.".
- Al listar las tarjetas del titular después del bloqueo, la tarjeta figura con estado `bloqueada`.

RF-G5-04 — Activar una tarjeta

- Dada una tarjeta bloqueada, al activarla la respuesta es 200 y `data.estado` es `activa`.
- Al bloquear y activar la misma tarjeta de forma alternada varias veces, todas las respuestas son 200 y el estado final es el de la última operación.
- Al activar una tarjeta en estado `vencida`, la respuesta es 200 y la tarjeta queda `activa`: el sandbox no impide la transición.
- Al activar una tarjeta inexistente, la respuesta es 404 `NOT_FOUND`.

8. Anexo normativo (BCP / SEDECO) y brechas

> Este anexo es material de referencia orientativo para dar contexto de negocio a los escenarios de prueba. Las normas citadas del Banco Central del Paraguay (BCP) y de la Secretaría de Defensa del Consumidor y el Usuario (SEDECO) se mencionan de forma general y deben validarse con su texto vigente antes de usarse como criterio de cumplimiento. El sandbox es un entorno didáctico y no pretende cumplir ninguna de ellas.

Referencia normativa	Expectativa	Estado en el sandbox
Ley 4595/2012 de tarjetas de crédito	El contrato debe fijar un límite de crédito explícito y las condiciones de uso antes de la emisión.	No implementado: la emisión de una tarjeta de crédito deja el límite vacío y no registra condición alguna.
Ley 4595/2012, bloqueo por denuncia del titular	El titular puede denunciar el extravío y el emisor debe bloquear el plástico de inmediato, dejando constancia de la denuncia.	Parcial: el bloqueo es inmediato, pero no se registra motivo ni denuncia, y cualquier portador de una API key válida puede bloquear cualquier tarjeta.
Estándares de la industria de medios de pago sobre datos de tarjeta	El número completo de la tarjeta no debe almacenarse ni exponerse; solo los últimos dígitos.	Cubierto: el sandbox nunca guarda un número real y solo devuelve el formato enmascarado.
Ley 1334/98 de Defensa del Consumidor (SEDECO), reversión de operaciones	El consumidor debe poder revertir la baja o el bloqueo y conocer el estado de su instrumento.	Cubierto en cuanto al estado: el listado muestra el estado actual y la activación revierte el bloqueo.
Marco del BCP sobre vencimiento y reemisión de instrumentos de pago	Una tarjeta vencida no puede volver a ponerse en servicio sin reemisión.	No implementado: activar una tarjeta vencida es una operación aceptada.

Brechas conocidas del sandbox

- Las transiciones de estado no validan el estado previo: una tarjeta vencida puede activarse.
- El estado `vencida` nunca se asigna automáticamente: no hay fecha de expiración en el modelo.
- La emisión no asigna límite de crédito ni distingue el tratamiento de una tarjeta de crédito respecto de una de débito.
- `saldo_actual` nunca cambia: no hay consumos, autorizaciones ni pagos que lo muevan.
- No hay tope de tarjetas por titular ni control de duplicados.
- El bloqueo no registra motivo ni solicitante, y no existe un historial de cambios de estado.
- `PATCH .../bloquear` y `.../activar` no filtran por `activo`: una tarjeta dada de baja con `DELETE` (RF-G5-07) ya no aparece en `GET`, pero igual puede bloquearse o activarse.
- El límite de crédito solo puede asignarse vía `PUT` (RF-G5-06), nunca en la emisión (RF-G5-02): una tarjeta de crédito recién emitida siempre nace sin límite.